Lab 13 – Code Refactoring Using AI Assistance

Name : Snehanjali
Hall Ticket No.: 2303A51305
Batch No.:05

Task 1: Refactoring – Removing Code Duplication Prompt
Refactor the following legacy Python code by removing duplicated logic and creating a reusable function to calculate the area and perimeter of rectangles. Add proper docstrings.

Legacy Code

Assignment13.py

```python
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

OUTPUT :

```
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

Refactored Code def
rectangle_details(length, width):
"""
Calculates and prints the area and perimeter of a rectangle.
Parameters:
length (int or float): Length of the rectangle
width (int or float): Width of the rectangle
""" area = length *
width perimeter = 2
* (length + width)
print("Area of
Rectangle:", area)
print("Perimeter of

Rectangle:", perimeter)
rectangle_details(5, 10)
rectangle_details(7, 12)
rectangle_details(10,
15)

```
Assignment13.py > rectangle_details
def rectangle_details(length, width):
    """
    Calculates and prints the area and perimeter of a rectangle.
    Parameters:
    length (int or float): Length of the rectangle
    width (int or float): Width of the rectangle
    """
    area = length * width
    perimeter = 2 * (length + width)
    print("Area of Rectangle:", area)
    print("Perimeter of Rectangle:", perimeter)
rectangle_details(5, 10)
rectangle_details(7, 12)
rectangle_details(10, 15)
```

OUTPUT :

```
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

Explanation

- Repeated code blocks were identified.
- A reusable function rectangle_details() was created.
- This improves readability and maintainability.
- If calculations change later, only the function needs modification.

Task 2: Refactoring – Optimizing Loops and Conditionals Prompt

Optimize the following code by replacing inefficient nested loops with a faster data structure such as a set.

Legacy Code

```
Assignment13.py > ...
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]

for s in search_names:
    found = False
    for n in names:
        if s == n:
            found = True

    if found:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
```

OUTPUT :

```
Charlie is in the list
Eve is not in the list
Bob is in the list
```

Refactored Code

names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
name_set = set(names) for name in
search_names: if name in name_set:
    print(f"{name} is in the list")
else:
  print(f"{name} is not in the list")

```
Assignment13.py > ...
search_names = ["Charlie", "Eve", "Bob"]

name_set = set(names)

for name in search_names:
    if name in name_set:
        print(f"{name} is in the list")
    else:
        print(f"{name} is not in the list")
```

OUTPUT :

```
Charlie is in the list
Eve is not in the list
Bob is in the list
```

Explanation

- Nested loops have **O(n²) complexity**.
- Using a **set** provides **O(1) lookup time**.
- This improves performance significantly for large datasets.

Performance Comparison

Method	Time Complexity
Nested Loop	O(n × m)
Set Lookup	O(n + m)

Task 3: Refactoring – Extracting Reusable Functions Prompt

Refactor the following script by extracting price and tax calculations into a reusable function called calculate_total(price).

Legacy Code

```
Assignment13.py > ...
print("Total Price:", total)

price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

OUTPUT :

```
Total Price: 295.0
Total Price: 590.0
```

Refactored Code

```
def calculate_total(price):
    """
    Calculates total price including tax.

    Parameters:     price (float): Base
price of the product
```

Returns:
float: Total price including tax
"""
tax = price * 0.18
total = price + tax
return total

print("Total Price:", calculate_total(250)) print("Total Price:", calculate_total(500))

```
Assignment13.py > ...
def calculate_total(price):
    """
    Calculates total price including tax.

    Parameters:
    price (float): Base price of the product

    Returns:
    float: Total price including tax
    """
    tax = price * 0.18
    total = price + tax
    return total

print("Total Price:", calculate_total(250))
print("Total Price:", calculate_total(500))
```

OUTPUT :

```
Total Price: 295.0
Total Price: 590.0
```

Explanation

- Repeated logic moved into **calculate_total()**.
- Improves **reusability** and **clean code structure**.
- Function can be reused across different modules.

Task 4: Refactoring – Replacing Hardcoded Values with Constants Prompt
Identify magic numbers in the code and replace them with named constants.

Legacy Code

```
Assignment13.py
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)
```

OUTPUT :

```
Area of Circle: 153.93791
Circumference of Circle: 43.98226
```

Refactored Code PI = 3.14159 RADIUS = 7
area = PI * (RADIUS ** 2) circumference = 2 *
PI * RADIUS print("Area of Circle:", area)
print("Circumference of Circle:", circumference)

```
Assignment11.3.py U    Assignment13.py U
Assignment13.py > ...
PI = 3.14159
RADIUS = 7

area = PI * (RADIUS ** 2)
circumference = 2 * PI * RADIUS

print("Area of Circle:", area)
print("Circumference of Circle:", circumference)
```

OUTPUT :

```
Area of Circle: 153.93791
Circumference of Circle: 43.98226
```

Explanation

- Hardcoded values are called **magic numbers**.
- Replacing them with **constants improves readability**.
- Constants allow **easy updates in one place**.

Task 5: Refactoring – Improving Variable Naming and Readability Prompt
Improve the readability of the following code by using descriptive variable names and comments.

Legacy Code

```
Assignment11.3.py U    Assignment13.py U ×
Assignment13.py > ...
a = 10
b = 20
c = a * b / 2
print(c)
```

OUTPUT:

```
100.0
```

Refactored Code

# Base and height of a triangle
base = 10 height = 20
# Formula for area of triangle triangle_area
= (base * height) / 2 print(triangle_area)

```
Assignment13.py > ...
# Base and height of a triangle
base = 10
height = 20

# Formula for area of triangle
triangle_area = (base * height) / 2

print(triangle_area)
```

OUTPUT :

```
100.0
```

Explanation

- Variables **a, b, c** were unclear.
- Renamed them to **base, height, triangle_area**.
- Added comments explaining the logic.